

ARCHITECTURE DECISION RECORD

ADR-001: MPS Authority, Scope, and Runtime Boundary

Natural-Language Treatment Planning System (NL-TPS)

Decision identifier: ADR-001

Decision version: 0.1

Date: 19 August 2026

Status: Provisionally accepted for controlled non-clinical implementation; authority cutover not approved

Related baseline: ENG-PKG-01 v0.3; AB-002, AB-003, AB-004, AB-008, and AB-011

STATUS AND SAFETY NOTICE

This decision controls an engineering and governance mechanism. It does not authorize patient data in MPS, clinical use, commissioning, plan approval, machine readiness, treatment transfer, or treatment delivery. Until the Stage C cutover criteria and named approvals are complete, the existing controlled source documents remain authoritative.

Contents

Document control	1
Decision summary	1
1 Context and problem statement	1
2 Decision	2
2.1 Role of MPS	2
2.2 Mechanism and policy separation	2
2.3 Runtime boundary	3
3 Authority transition	3
4 Initial language family	4
5 Typed trace graph and deterministic derivation	4
6 PlanIntent definitions, quantities, and release profiles	5
7 MPS engineering controls	5
7.1 Structural agent access	5
7.2 Persistence, version control, and migration	6
7.3 Reproducible build and CI	6
8 Initial implementation increment	7
8.1 Four-language bootstrap	7
8.2 119-HLR mirror importer	7
9 Alternatives and trade-off decision	7
10 Risks and mitigations	8
11 Verification and acceptance	9
12 Rollback and supersession	10
13 Official implementation references	10

Document control

Field	Controlled value
Decision owner	Architecture Board - proposed; named accountable owner and alternates require governance recording
Required reviewers	Systems engineering, clinical governance, quality, independent V&V, safety/risk, cybersecurity, privacy, configuration management, software architecture, and affected clinical representatives
Decision scope	Offline specification authoring, semantic validation, generation, migration, and governed release packaging
Excluded scope	Clinical runtime, patient transactions, patient data, dose calculation, treatment delivery, professional decisions, executed evidence, and systems-of-record authority
Supersedes	None; this decision refines ENG-PKG-01 AB-002 and partially resolves D-ENG-006 for the governance-compiler role only
Recorded approvals	None in this document; provisional status is not evidence of named institutional approval
Next review	Before Stage B equivalence begins and before any MPS-generated artifact is used as an approved runtime input

Decision summary

JetBrains MPS shall be adopted as the proposed **offline governance compiler** for NL-TPS. It shall model governed semantics, validate structural and domain constraints, and generate reviewable and machine-consumable artifacts. It shall not be placed in the patient-specific runtime path, hold patient instances, perform clinical calculations, make professional decisions, or serve as an online dependency of the deterministic safety kernel.

Authority shall move only through four controlled stages: current documents to an MPS mirror; demonstrated equivalence; an explicitly approved governance cutover; and, only after cutover, generation of controlled human-readable and runtime artifacts from the MPS model. The present decision authorizes Stage A engineering and preparation for Stage B. It does not declare Stage C complete.

The runtime shall consume only immutable, versioned, integrity-verified release bundles generated from an approved model baseline. Safety-critical mechanisms remain hand-maintained and independently verified. MPS supplies approved policy and semantic data; it does not generate arbitrary clinical behavior.

1 Context and problem statement

The NL-TPS corpus is already a large, interconnected assurance system: requirements, derived interface obligations, components, team allocations, risks, trade studies, V&V claims, accreditation profiles, and implementation decisions. Much of the current prose is mechanically expanded, while the high-value engineering information lies in stable identifiers, normative statements, typed

relationships, applicability, ownership, authority, and evidence expectations.

The current Overleaf and source artifacts are human-reviewable and configuration controlled, but they do not by themselves prevent invalid trace relationships, inconsistent units, duplicate identifiers, unsupported lifecycle transitions, or divergence among repeated representations. Conversely, immediately declaring a new model authoritative would break the existing document-control chain before equivalence and rollback are proven.

MPS is suited to direct abstract-syntax-tree authoring through concepts, properties, references, children, constraints, typesystems, editors, and generators. That capability is useful only if it is bounded from clinical runtime authority and introduced through controlled migration.

2 Decision

2.1 Role of MPS

MPS shall provide the engineering and governance plane for:

- stable governed identifiers, versions, lifecycle state, provenance, and external references;
- requirements, hazards, risk controls, decisions, approval requirements, release gates, and typed traceability;
- definitions of PlanIntent, authority, roles, modes, state transitions, computable rules, physical quantities, units, and commissioned-use envelopes;
- components, interfaces, teams, allocations, V&V claims, evidence requirements, adapters, and release profiles;
- structural validation, domain validation, deterministic derivation, document generation, runtime-schema generation, trace reports, migration, and release-bundle assembly.

MPS shall not provide:

- interactive availability required for an NL-TPS patient workflow;
- storage of patient images, registrations, structures, dose, plans, or patient-specific PlanIntent instances;
- prescription, approval, signature, attestation, QA waiver, return-to-service, release, transfer, or delivery authority;
- an independent dose engine, optimizer, DICOM system of record, audit ledger, or executed-evidence repository;
- a bypass around the deterministic Zone 1 kernel or token-gated Zone 2 execution boundary.

2.2 Mechanism and policy separation

Safety mechanisms shall remain hand-maintained deterministic runtime code with independent tests and review. The initial mechanism set includes:

- `authorize()`, `assert_context()`, and `validate_state_transition()`;
- `validate_release_profile()` and `validate_commissioned_envelope()`;
- `validate_required_approvals()` and `invalidate_downstream()`;
- `authorize_export()` and `fail_closed()`.

MPS may define and generate versioned policy consumed by those mechanisms: who may request which action, in which mode and state, for which profile and object version, under which dependencies and evidence, and with which approval requirements. Generated policy never expands the runtime mechanism's compiled allowlist and never grants A4 authority to AI or automation.

2.3 Runtime boundary

The engineering boundary is:

```
MPS model baseline → validate → generate → review/approve → hash/sign
release bundle
runtime boundary
release bundle → deterministic loader → Zone 1 kernel → capability-gated Zone
2 adapters → Zone 3 systems
```

No runtime component shall query a live MPS repository or accept an unapproved working model. The bundle identifier and content hashes shall be bound into the runtime configuration, `KernelDecisionRecord`, `ExecutionCapability` where applicable, and audit evidence.

3 Authority transition

Stage	Authority	Permitted work	Exit control
A - Mirror	Existing controlled source documents	Import without altering source meaning; create typed nodes and links; report exceptions	Exact expected counts; unique IDs; source hashes; no silent conflict resolution; reproducible import
B - Equivalence	Existing controlled source documents	Compare IDs, exact normative text, domains, methods, allocations, risks, V&V, and generated views	Approved equivalence report; zero unexplained discrepancy; tested round trip; rollback rehearsal
C - Governance cutover	Changes only after named cutover approval	Baselined MPS model becomes authoritative for explicitly listed structured normative content	Change record; accountable approvals; effective date; repository and document-control updates; training; contingency and rollback acceptance
D - Generation	Approved MPS baseline for the cutover scope	Generate documents, schemas, policies, profiles, trace matrices, and test manifests	Reproducible CI; signed manifest; independent release verification; generated-source drift gate

Authority is field- and scope-specific. Executed test results remain authoritative only in the approved evidence repository; patient artifacts remain in clinical systems or approved runtime stores; dose remains attributable to the commissioned calculation source; professional decisions remain attributable to authenticated qualified people.

4 Initial language family

The first MPS project shall contain four composable languages. The names and first responsibility boundaries are controlled by this decision; detailed concept design remains subject to model review.

Language	Owns	Initial concepts and invariants
<code>nltps.foundation</code>	IDs, versions, lifecycle, authority vocabulary, quantities, units, provenance, and external references	GovernedElement, StableId, Version, LifecycleState, AuthorityClass, PhysicalQuantity, Unit, ProvenanceRef; globally unique IDs and dimensionally valid quantities
<code>nltps.governance</code>	Requirements, hazards, controls, approvals, decisions, release gates, and traceability	Requirement, RequirementPattern, DerivedRequirement, Hazard, RiskControl, Decision, ApprovalGate, TraceLink; relation-specific source/target constraints
<code>nltps.clinicalintent</code>	Planning-intent definitions, roles, permissions, modes, state transitions, computable rules, and commissioned scope	PlanIntentDefinition, ActionDefinition, Role, Permission, OperatingMode, StateTransition, ComputableRule, ReleaseProfile, CommissionedUseEnvelope; A4 is not representable as AI authority
<code>nltps.realization</code>	Components, interface families, teams, allocations, V&V claims, evidence expectations, and adapters	Component, InterfaceFamily, InterfaceContract, Team, Allocation, VerificationClaim, ExecutableSuiteRef, ManualEvidenceRef, ExternalSystem, AdapterCapability; every realized obligation has an owner and verification binding

Dependency direction shall be acyclic: foundation is independent; governance depends on foundation; clinicalintent depends on foundation and governance vocabulary where necessary; realization depends on foundation and governance and may reference clinicalintent definitions. Domain extensions such as proton, Machine QA, accreditation, and disease-site profiles shall extend these languages without moving their specific policy into the universal foundation.

5 Typed trace graph and deterministic derivation

TraceLink shall be a governed node, not free text. The initial relation vocabulary shall include DERIVES_FROM, DECOMPOSES, ALLOCATED_TO, REALIZED_BY, MITIGATES, VERIFIED_BY, EVIDENCED_BY, DEPENDS_ON, INVALIDATES, SUPERSEDES, APPLIES_TO, and APPROVED_BY. Each relation shall con-

strain allowed source and target concepts, lifecycle state, cardinality where applicable, rationale, status, and version.

Mechanical decomposition shall be represented by controlled patterns. For each applicable HLR, the interface pattern may derive exactly three obligations - contract, runtime behavior, and evidence/conformance - with deterministic stable identifiers. Derived nodes shall not contain independently editable duplicate prose except through a controlled, justified `RequirementOverride`. Generation must preserve the original HLR and existing child identifiers during the migration period.

6 PlanIntent definitions, quantities, and release profiles

MPS shall define the PlanIntent language and generate its schema and typed bindings. Patient-specific PlanIntent instances shall live in the runtime record and audit system, never in the governance model. The language shall make ambiguity, unresolved information, conflicts, expected outputs, basis, context, and version bindings explicit; professional approval effects shall be excluded from AI-creatable intent.

Physical quantities shall be typed rather than represented as arbitrary strings. Dose, RBE-weighted dose, length, energy, fraction count, percentage, and dimensionless values shall have explicit units and conversion policy. Dimensionally invalid comparisons or implicit conversion between physical and RBE-weighted dose shall fail model validation.

`ReleaseProfile` is the modeled unit approved for runtime consumption. A generated release bundle shall, as applicable, contain:

- `release-profile.json`, `intent.schema.json`, and typed-binding manifests;
- `safety-policy.json`, `evidence-profile.json`, and `commissioned-envelope.json`;
- `adapter-capabilities.json`, `model-registry.json`, and approved monitoring configuration;
- `requirements.json`, `traceability.json`, and `vv-manifest.json`;
- `hashes.json`, build provenance, generator versions, MPS version, model-baseline ID, approval references, and signature metadata.

The runtime loader shall reject a missing, altered, expired, unsigned when signature is required, incompatible, unapproved, or out-of-scope bundle.

7 MPS engineering controls

7.1 Structural agent access

The MPS Projectional Agent Toolkit may be used only as an engineering aid. It is not required to build or operate NL-TPS. Because JetBrains labels the toolkit experimental, its output is untrusted until normal model validation, tests, review, and CI succeed.

Controls are:

- no direct text editing of MPS persistence XML;
- expose only necessary `mps_mcp_*` tools and bind the service locally;
- no patient data, production credentials, signing keys, or clinical-system access in the MPS project or agent context;
- all agent changes are attributable, reviewed through Git, and subject to the same checks as human changes;
- an agent cannot approve its own change, alter recorded professional authority, or suppress validation findings;
- pin the MPS and plugin versions for each governed build.

JetBrains documents a current toolkit limitation: if any open MPS project path contains a space, MCP calls fail. The present repository path contains spaces. Live agent-assisted MPS work shall therefore use an approved space-free working path or other verified vendor-supported remedy, while preserving a single reviewed Git history. This ADR does not authorize moving the repository or creating an unmanaged duplicate.

7.2 Persistence, version control, and migration

Normative root models shall use file-per-root persistence unless a documented exception is approved. This reduces merge scope and makes identifier-level review practical. Small enumerations and closely coupled configuration roots may remain grouped when that is the clearer review unit.

A change to a language concept, constraint, typesystem rule, editor contract, generator, or migration is a governed product change. It requires a language-version decision, migration script where needed, migration tests, model impact report, regeneration, regression, and baseline approval. Generated output shall never be edited as the source of a normative change.

7.3 Reproducible build and CI

The release pipeline shall run without an interactive MPS session and shall:

1. load the pinned MPS project and dependencies;
2. check all models and fail on errors;
3. run node, constraints, typesystem, generator, editor where applicable, and migration tests;
4. generate derived models and release artifacts deterministically;
5. compare generated output with committed or approved snapshots;
6. run runtime schema, kernel, trace-completeness, and architecture-invariant tests;
7. assemble a release manifest, calculate hashes, and submit the candidate bundle for independent review and signing.

CI shall fail on duplicate or orphan identifiers, invalid trace endpoints, missing owner or allocation, risk controls without hazards or evidence expectations, V&V claims without execution/manual bind-

ings, invalid unit relations, broken references, unapproved evidence sources, missing commissioned-use scope, prohibited write capability in a default profile, generated drift, unapplied migration, or an unapproved release profile. Structural PASS never implies clinical V&V PASS.

8 Initial implementation increment

8.1 Four-language bootstrap

The repository shall hold an implementation-neutral bootstrap specification for the four languages, their dependency direction, initial concepts, constraints, generators, tests, and acceptance criteria. An MPS-aware engineer or structurally connected agent shall materialize that specification in the live MPS repository. Generated `.mps` serialization shall be produced by MPS and shall not be hand-authored.

8.2 119-HLR mirror importer

The first importer shall read the controlled HLR LaTeX source and produce a deterministic, schema-validated intermediate bundle containing all 119 requirements. For each record it shall preserve the stable ID and exact LaTeX normative text, provide normalized review text, carry the domain, source/hazard field, verification methods, source line, and content hash, and record the source-file hash.

Importer acceptance criteria are:

- exactly 119 unique records and exact domain counts of GOV 7, SAF 10, NLI 8, EVD 8, CLN 9, PLN 12, REV 9, DAT 10, AIM 8, HFE 6, SEC 8, OPS 7, VAL 9, and ACC 8;
- sequential IDs within each domain and no unrecognized verification method;
- every normative statement contains “shall” and retains its exact source field;
- repeated execution against unchanged source is byte-for-byte reproducible;
- check mode detects source or output drift and returns failure;
- MPS import creates one governed HLR root per record under file-per-root persistence, creates no derived HLIR/SIR node in this increment, and reports rather than resolves conflicts;
- a reverse export and comparison demonstrate ID, exact-text, domain, method, and record-count equivalence before Stage B exit.

The full 357/1,071 derived interface tree, components, teams, risk controls, V&V claims, and profiles shall be imported only after the four-language metamodel and HLR mirror pass review. This keeps the first increment small enough to diagnose semantics rather than merely move document volume.

9 Alternatives and trade-off decision

Alternative	Benefits	Principal limitations	Disposition
Keep LaTeX/Word as the permanent semantic source	Familiar review and immediate Overleaf publication	Weak structural constraints; repeated prose and trace drift remain expensive	Retain as authority during Stages A/B; do not select as the target semantic compiler
Use plain YAML/JSON as the sole source	Simple CI and broad tooling	Relationships, domain editors, migrations, and semantic constraints become custom application code	Retain generated interchange and review formats; reject as sole authoring model
Use a conventional requirements database only	Mature records, workflows, and reporting	Limited domain-language composition, type rules, and generated runtime contracts without extensive customization	May remain an integration or evidence system; not selected for semantic compilation
Place MPS in the clinical runtime	Immediate access to live model semantics	Adds interactive IDE/toolchain availability, attack surface, and uncontrolled working-state risk to patient workflows	Rejected
MPS offline with staged cutover and immutable bundles	Typed graph, projectional editing, constraints, generation, migration, and clean runtime separation	Specialized skills, toolchain pinning, migration burden, and experimental-agent limitations	Selected provisionally for controlled non-clinical implementation

10 Risks and mitigations

Risk	Failure condition	Required mitigation and evidence	Release effect
ADR1-R-01	Silent authority switch creates conflicting sources of truth	Four-stage transition, field-level authority register, exact equivalence, named cutover, rollback rehearsal	Blocks Stage C
ADR1-R-02	Model or generator defect produces unsafe policy	Typed constraints, negative corpus, generator snapshots, independent policy review, runtime mechanism allowlist, fail-closed loader	Blocks bundle approval
ADR1-R-03	Generated artifact is altered or stale	Deterministic build, content hashes, signed manifest, drift check, runtime hash binding	Runtime rejects bundle
ADR1-R-04	MPS becomes a runtime single point of failure	Offline-only architecture; committed approved bundle; tested rollback and continuity	Prohibits live MPS dependency

Continued on next page

Risk	Failure condition	Required mitigation and evidence	Release effect
ADR1-R-05	Agent corrupts model or expands scope	Structural tools only, minimum exposure, local bind, no PHI/credentials, Git review, validation, CI, separation of approval	Change rejected
ADR1-R-06	Experimental toolkit or space-containing path causes unreliable edits	Pin version; approved space-free work path; non-agent workflow remains available; no direct XML edit	Blocks agent-assisted work, not model build
ADR1-R-07	MPS specialist dependency exceeds program capacity	Four-language limit, documented concepts, paired ownership, training, reproducible headless build, succession plan	Blocks Stage C if unsupported
ADR1-R-08	Language evolution invalidates governed baselines	Versioned language, migrations, migration tests, impact analysis, old-bundle retention and replay	Blocks baseline change
ADR1-R-09	File-per-root creates noisy or misleading review	Stable roots, logical grouping exceptions, semantic diff/merge checks, reviewer training	Requires remediation before merge
ADR1-R-10	Structured consistency is mistaken for clinical correctness	Explicit evidence-state semantics; no inferred PASS; independent clinical, physics, HFE, commissioning, and V&V records	Prohibits clinical claim

11 Verification and acceptance

ADR-001 shall be considered implemented for Stage A only when:

1. this ADR, ENG-PKG-01, and the machine-readable decision and architecture records agree on scope and authority;
2. the four-language bootstrap manifest passes dependency and concept-ownership checks;
3. the HLR import bundle is reproducibly generated and validates exactly 119 records with the controlled domain distribution;
4. no MPS persistence XML was hand-authored and the live-model materialization procedure is ready for structural execution in MPS;
5. the current controlled documents remain visibly authoritative and no runtime consumes the mirror;
6. source, generated artifacts, tools, and reports are version controlled and reviewed.

Stage B and C require separate recorded evidence. Stage C cannot be inferred from an accepted ADR title, an imported model, a passing structural test, a generated PDF, or an architecture-board recommendation.

12 Rollback and supersession

Before Stage C, rollback is deletion or archival of the non-authoritative MPS mirror and continuation from the unchanged controlled documents. After Stage C, rollback shall use the last approved MPS model baseline and last approved signed release bundle, with an impact-assessed change record; reverting to document authority requires an explicit authority decision and reconciliation of changes made after cutover.

This ADR may be superseded only by a later decision that explicitly identifies the replacement governance compiler, authority model, migration evidence, runtime boundary, affected baselines, and rollback path.

13 Official implementation references

These references describe MPS capabilities and limitations; they do not constitute clinical validation or supplier approval.

- [JetBrains MPS, *Structure*](#).
- [JetBrains MPS, *Constraints*](#).
- [JetBrains MPS, *Typesystem*](#).
- [JetBrains MPS 2026.1, *Projectional Agent Toolkit*](#).
- [JetBrains MPS, *Project tool window and persistence*](#).
- [JetBrains MPS, *Build Language*](#).
- [JetBrains MPS, *Testing languages*](#).
- [JetBrains MPS, *Migrations*](#).

Approval record

Accountability	Name	Decision/date	Conditions or dissent
Architecture decision owner	TBD	Not recorded	Provisional non-clinical direction only
Clinical governance	TBD	Not recorded	Required before authority cutover
Quality/configuration management	TBD	Not recorded	Required before authority cutover
Independent V&V	TBD	Not recorded	Equivalence evidence review required
Cybersecurity/privacy	TBD	Not recorded	Required before agent tooling or governed build environment approval